# Functions and Variables

# 1. Hello World

```
Hello, World!
```

# Jupyter notebook on Ed

- **Cells**: code cells run Python; text cells hold the instructions
- **Run a cell**: Click ▶️ or `Ctrl+Enter`
- **Order matters**: cells share variables, so run them from top to bottom
- **Editor**: autocomplete and syntax highlighting help you spot typos
- **Start over**: `...` › `Reset to Scaffold` brings back the starter code (your changes will be lost)

# Anatomy of `print()`

```
print("Hello, World!")
```

- **Function**: a named block of code that does a task
  - `print()` : prints the argument to the screen
- **Argument**: input to the function (in parentheses)
  - `"Hello, World!"`
- **Side effect**: what the function does to the outside world
  - print to the screen

# Bugs 🐞

```
print("Hello, World!"
```

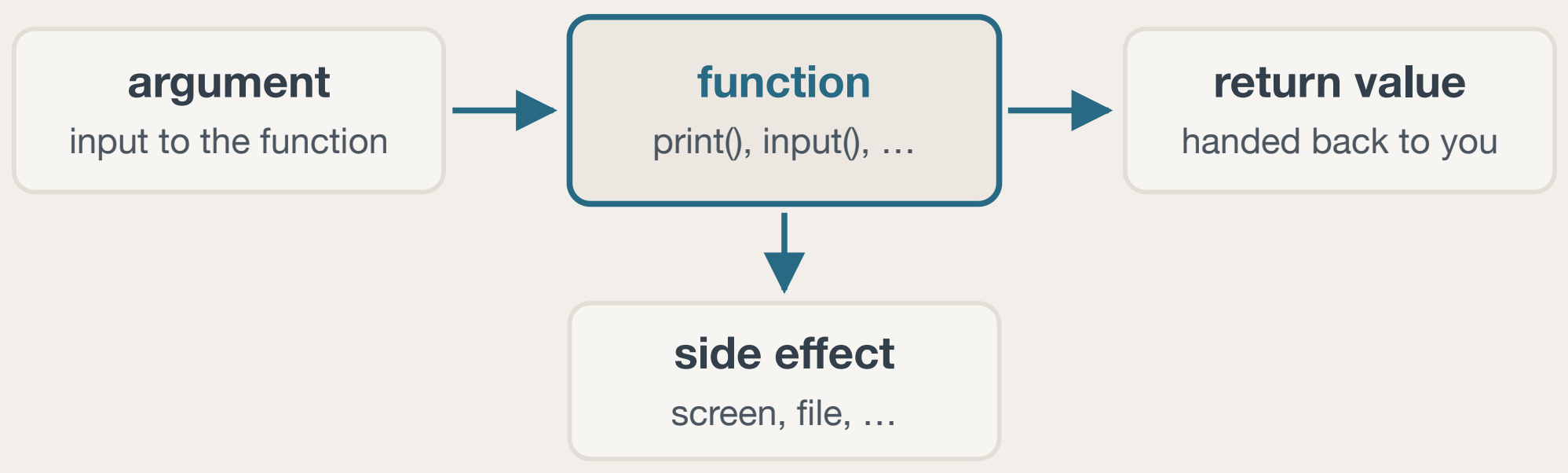
```
  Cell In[1], line 1
    print("Hello, World!"
                          ^
SyntaxError: incomplete input
```

# 2. Hello to You

```
What's your name? John
Hello, John
```

# Anatomy of `input()`

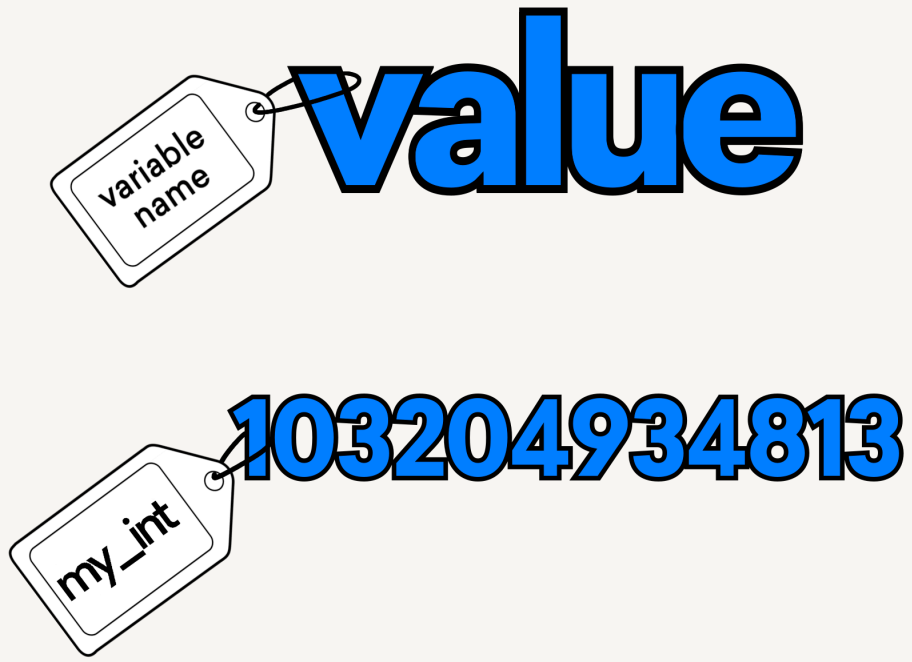
```
print(input("What's your name? "))
# function:       input() to get user input
# argument:       "What's your name? "
# side effect:    shows the prompt and waits for typing
# return value:   what the user typed, handed to print()
```



# Variables: name tags for values

```
name = "John"    # the tag name points to a value
print(name)      # John
name = "Emily"   # the tag moves to a new value
print(name)      # Emily
```

`=` **assigns**: work out the right side, then attach the name on the left



# 🤔 Printing a variable

```
answer = input("What's your name? ")  # store user input in a variable

print(answer)

print("Hello, answer")
```

# `+` or an f-string to join strings and variables

```
answer = input("What's your name? ")   # user types John

# +: joins two strings; the space goes inside "Hello, "
greeting = "Hello, " + answer
print(greeting)                        # Hello, John

# f-string: f before the quote; {answer} is replaced by its value
greeting = f"Hello, {answer}"
print(greeting)                        # Hello, John

print("Hello, {answer}")               # forgot the f: Hello, {answer}
```

# Printing multiple arguments

```
answer = input("What's your name? ")   # user types John

print("Hello,", answer)     # Hello, John: print() puts a space between arguments
print("Age:", 25)           # Age: 25: arguments can be numbers, no str() needed
```

# `#` to write comments that Python ignores

```
# + operator
print("Hello, " + answer)

# f-string
print(f"Hello, {answer}")

# multiple arguments
print("Hello,", answer)
```

# 💻 3. Self-introduction

**Requirements:**

- Use `input()` to ask for the user's name and age, and store each in its own variable.
- Use `print()` to display `Hello, my name is [name]. I am [age] years old.`

**Expected Outputs:**

```
What's your name? Emily
What's your age? 25
Hello, my name is Emily. I am 25 years old.
```

# 4. Messy input

```
What's your name?     john
Hello, John

What's your name? jAnE doE
Hello, Jane Doe
```

# `strip()` to remove surrounding spaces

`strip()` removes leading and trailing whitespace from a string and returns a new string

```
answer = input("What's your name? ")   # user types "   john  "

print(answer.strip())          # "john": strip() returns a cleaned copy
print(answer)                 # "   john  ": answer is unchanged

answer = answer.strip()       # point answer at the cleaned copy
print("Hello, " + answer)     # Hello, john
```

# Methods: functions that belong to an object

```
name = "   John   "     # name → "   John   "; string object
print(name.strip())     # "John": strip() is a method of string objects
number = 871284         # number → 871284; integer object
print(number.strip())   # AttributeError: 'int' object has no attribute 'strip'
```

`"   John   "` is a **string object**

`name.strip()` calls the strip method on that object

`871284` is an **integer object**

`number.strip()` fails because integers don't have a strip method

---

https://docs.python.org/3/library/stdtypes.html#string-methods

# `capitalize()` and `title()` to fix letter case

```
answer = input("What's your name? ")    # user types "   jAnE doE  "

stripped = answer.strip()               # "jAnE doE"
capitalized = stripped.capitalize()    # "Jane doe"
print(capitalized)                     # Jane doe

print(answer.strip().capitalize())     # Jane doe: first letter only

print(answer.strip().title())          # Jane Doe: first letter of each word

print(answer.title())                  # "   Jane Doe  ": title() doesn't strip
```

**chaining methods:** `answer.strip().capitalize()` calls `strip()` first, then `capitalize()` on the result since `strip()` returns a string

# 5. Hello Function

```
hello()
# Output: Hello, world

hello("John")
# Output: Hello, John
```

# `def` to define a function

```
def hello():                  # def, a name, (), and a colon
    print("Hello, world")     # body: indented (a tab), so it belongs to hello()
print("Bye")                  # not indented: not part of hello()
hello()                       # call: it runs the code in hello()
```

**Defining stores the code. Calling runs it.**

- 1st line: Python first registers the function without running its body.
- 3rd line: It runs the print("Bye") line, because it's part of the main program (not indented), not the function.
- 4th line: It calls hello(), which runs the code in the function body.

# Parameters to accept arguments

```
def hello():        # no parameters
    print("Hello, world")

hello("John")       # TypeError: hello() takes 0 positional arguments but 1 was given
```

```
def hello(name):            # name: parameter
    print("Hello,", name)

# Call the function with an argument
hello("John")               # name = "John"
hello("Emily")              # name = "Emily"
```

- **Parameter**: a placeholder in the definition ( `name` )
- **Argument**: the value passed in the call ( `"John"` )

# Parameters with default values

```
def hello(name):           # name: parameter
    print("Hello,", name)

hello()                    # TypeError: hello() missing 1 required positional argument: 'name'
```

```
def hello(name="world"):  # name: parameter with default value "world"
    print("Hello,", name)

# Call the function with an argument
hello("John")   # name = "John" -> Hello, John

# Call the function without an argument; default value is used
hello()         # name = "world" -> Hello, world
```

# Positional vs. keyword arguments

```
def greet(greeting, name):
    print(greeting + ",", name)

# positional: matched by order
# 1st argument → 1st parameter, 2nd → 2nd
greet("Hello", "John")      # greeting = "Hello", name = "John" → Hello, John
greet("John", "Hello")      # greeting = "John", name = "Hello" → John, Hello

# keyword: matched by parameter name, so order doesn't matter
# 1st argument → greeting, 2nd → name
greet(name="John", greeting="Hello")    # → Hello, John
```

# `help()` to look up function details

```
help(print)
```

```
print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
      string inserted between values, default a space.
    end
      string appended after the last value, default a newline.
```

```
print("Hello", "John")             # Hello John  (sep defaults to " ")
print("Hello", "John", sep="-")    # Hello-John  (keyword argument)
```

https://docs.python.org/3/library/functions.html#print

# 🤔 What does each call print?

```
def hello(a, b="Doe"):
    print("Hello", a, "and", b)
# 1
hello("John", "Doe")
# 2
hello("Doe", "John")
# 3
hello(b="Doe", a="John")
# 4
hello("John")
# 5
hello(b="John")
```

# `return` to hand a value back to the caller

**caller:** where the function is called

```
def hello_message(name):
    msg = "Hello, " + name
    return msg                          # return the value of msg to the caller

message = hello_message("John")  # the caller; message = "Hello, John"
print(message)
```

**No `return`?** The function hands back `None`, Python's value for "nothing".

## 🤔 Scope: can `hello()` see `main()`'s variables?

```
def main():
    username = input("What's your name? ")
    hello()

def hello():
    print("Hello,", username)

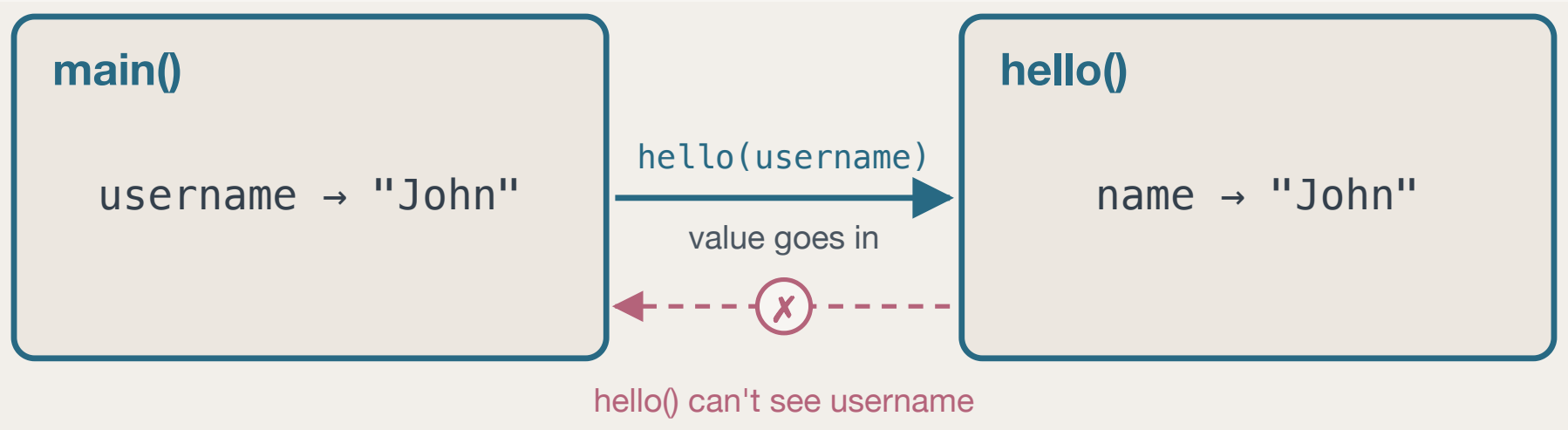
main()
```

# Fixing scope by passing arguments

```
def main():
    username = input("What's your name? ")
    hello(username)   # name = username

def hello(name):      # parameter
    print("Hello,", name)

main()
```



# Top-down design: `main()` for the flow, functions for the details

```
# main: WHAT the program does, step by step
def main():
    username = input("What's your name? ")   # 1. ask for a name
    hello()                                  # 2. say hello

# hello: HOW to say hello (write it later)
def hello():
    ...

main()   # call main to start the program
```

# 🤔 What gets printed?

```
def hi_john(name):
    name = "John"
    print("Hi,", name)
def hi_doe(name):
    name = name + " Doe"
    return "Hi, " + name
name = "Jane"
```

```
# 1
hi_john(name)
# 2
print(name)
# 3
x = hi_john(name)
print(x)
# 4
hi_doe(name)
# 5
print(hi_doe(name))
```

# 💻 6. Self-introduction 2

**Requirements:**

- Define `ask_name()`, which asks for the user's name with `input()` and returns it.
- Define `ask_age()`, which asks for the user's age and returns it.
- Define `introduce_message()`, which takes `name` and `age` as parameters and returns the string `Hello, my name is [name]. I am [age] years old.`
- `main()` is already written: it calls your functions in order and prints the message.

**Expected Outputs:**

```
What's your name? Emily
What's your age? 25
Hello, my name is Emily. I am 25 years old.
```

# 7. Calculator

```
Enter a number: 5
Enter another number: 3
8
```

# 🤔 Adding two inputs

```
def calculator():
    x = input("Enter a number: ")
    y = input("Enter another number: ")
    z = x + y

    print(z)

calculator()
```

# `int()` to convert string to integer

```
def calculator():
    x = input("Enter a number: ")
    y = input("Enter another number: ")
    z = int(x) + int(y)   # convert strings to integers before adding

    print(z)

calculator()
```

# `type()` to check a value's type

```
x = input("Enter a number: ")
type_x = type(x)
print(type_x)         # <class 'str'>

y = int(x)
type_y = type(y)
print(type_y)         # <class 'int'>
```

## `int()`, `float()`, `str()` to convert between types

```
int("25")          # 25       string → integer
float("2.5")       # 2.5      string → decimal number
str(25)            # "25"     number → string

int("2.5")         # ValueError: use float() for decimals
```

# `round()` to limit decimal places

```
round(2 / 3, 2)     # 0.67   2 decimal places
round(2.9)          # 3      no second argument: nearest whole number
int(2.9)            # 2      int() cuts off; it doesn't round
```

# 💻 8. Self-introduction 3

**Requirements:**

- Define `ask_birthyear()`, which asks for the user's birth year and returns it.
- Define `calc_age()`, which takes the birth year as an integer and returns the age in 2026.
- `ask_name()` and `introduce_message()` from Exercise 6 are already provided.
- Complete `main()`: fill in the blanks to convert the birth year to an integer and the age back to a string.

**Expected Outputs:**

```
What's your name? Emily
What's your birth year? 1998
Hello, my name is Emily. I am 28 years old.
```

# Wrap up

- Variable: a name tag for a value; `=` assigns
- Function: a named block of code; arguments in, side effect and/or return value out
- Method: a function that belongs to a value, e.g., `answer.strip()`
- `def`: define a function; parameters receive the arguments of a call
- Scope: names stay inside their function; pass values in as arguments
- `return`: hand a value back to the caller
- `int()`, `float()`, `str()`: convert between types